



Tài Liệu Hệ Thống API Backend - Tiên Nga Group (Chi Tiết Logic & Công Thức)

Chào mừng bạn đến với tài liệu kỹ thuật chi tiết của hệ thống API thuộc dự án **Backend Bot & Quản Lý Vận Hành Tiên Nga**. Tài liệu này mô tả chi tiết về **Tổng quan, Mục đích sử dụng, các Ràng buộc logic nghiệp vụ, Thuật toán và Công thức tính toán cụ thể** của từng phân hệ.

1. Phân Hệ Xác Thực & Phân Quyền (Authentication & Employee Management)

1.1. Tổng quan về API

Quản lý người dùng, hồ sơ nhân viên và cấp phát quyền hạn truy cập (Role-Based Access Control).

- **Các Endpoint chính:**
 - `POST /api/v1/auth/register` : Đăng ký tài khoản (hệ thống mã hóa mật khẩu trước khi lưu).
 - `POST /api/v1/auth/login` : Xác thực tài khoản và cấp JWT token.
 - `PUT /api/v1/auth/update-permissions/{employee_id}` : Cập nhật quyền hạn nhân viên (Admin).
 - `GET /api/v1/employee/get-employee` : Lấy danh sách nhân sự kèm bộ lọc.

1.2. Logic Nghiệp Vụ & Ràng buộc

- **Mã hóa mật khẩu:** Sử dụng thuật toán băm mật khẩu một chiều (ví dụ: `bcrypt` hoặc `pbkdf2_sha256`) để bảo mật.
 - **Thời hạn token:** Access Token được cấp dưới dạng JWT ký bằng khóa bí mật (`SECRET_KEY`), thời gian hết hạn được định nghĩa bởi `Auth.ACCESS_TOKEN_EXPIRE_MINUTES` trong `appsettings.json` (mặc định là 1440 phút - 24 giờ).
 - **Quyền hạn truy cập:** Sử dụng Dependency Injection trong FastAPI:
 - `current_user: Credential = Depends(require_permission("tien-nga"))` -> Yêu cầu tài khoản có quyền vận hành Tiên Nga.
 - `current_user: Credential = Depends(require_permission("attendance"))` -> Yêu cầu tài khoản có quyền chấm công/lương.
-

2. Phân Hệ Chấm Công & Tính Lương (Attendance & Payroll)

2.1. Tổng quan về API

Quản lý ngày công, giờ làm việc thực tế, tính toán lương dự thảo và quản lý bảng lương chính thức.

- **Các Endpoint chính:**
 - `GET /api/v1/get-attendance` : Lấy lịch sử chấm công theo tháng của nhân viên.

- GET `/api/v1/get-salaries` : Dự thảo bảng tính lương hàng tháng (Draft).
- POST `/api/v1/add-payrolls` : Kết khóa và xuất bảng lương chính thức (Bulk).
- DELETE `/api/v1/delete-payrolls` : Xóa bảng lương đã khóa và hoàn trả số dư nợ lương của nhân viên.

2.2. Logic Nghiệp Vụ & Công thức Tính toán

A. Tính Ngày Công Tiêu Chuẩn (Standard Days)

Dựa theo tháng, năm và chế độ làm việc (`work_type` của nhân viên, ví dụ: 3 - nghỉ Chủ nhật, hoặc các cấu hình khác). Hệ thống sử dụng hàm quét từng ngày trong tháng để loại trừ các ngày nghỉ không tính công:

$$\text{Ngày công chuẩn} = \sum_{d=1}^{\text{Số ngày trong tháng}} 1 \quad (\text{nếu ngày } d \text{ là ngày làm việc theo } work_type)$$

B. Tính Ngày Công Thực Tế (Actual Workdays)

Quét toàn bộ bản ghi chấm công của nhân viên trong tháng:

- Với mỗi ngày có quét vân tay/nhập công (`check_in_time` is not None hoặc `working_time` > 0):
 - Nếu ngày đó được tích chọn nửa ngày (`is_half_day` = True), cộng thêm 0.5 công.
 - Ngược lại, cộng thêm 1.0 công.

$$\text{Ngày công thực tế} = \sum (\text{Ngày công thường} \times (0.5 \text{ nếu nửa ngày, } 1.0 \text{ nếu cả ngày}))$$

C. Tính Lương Làm Thêm Giờ (Overtime Amount)

- Nếu có nhập tay trường `overtime` (giờ tăng ca): Lấy trực tiếp giá trị đó.
- Nếu chấm công qua máy:

$$\text{Số giờ tăng ca} = \frac{\text{Thời gian kết thúc} - \text{Thời gian bắt đầu}}{3600 \text{ giây}}$$

$$\text{Lương tăng ca} = \text{Tổng số giờ tăng ca} \times \text{Đơn giá lương tăng ca của nhân viên}$$

D. Công thức Tính Lương Nhận Thực Tế (Draft & Official)

$$\text{Lương nhận} = \left(\frac{\text{Lương CB} + \text{Lương tăng ca} + \text{Phụ cấp ăn trưa} + \text{Phụ cấp hiệu suất} + \text{Phụ cấp khác}}{\text{Ngày công chuẩn}} \times \text{Ngày công thực} \right) + \text{Thưởng} - \text{BHXH} - \text{Phạt đi trễ}$$

E. Cơ chế Cập nhật Nợ Lương (total_debt)

- Khi **khóa bảng lương** (POST `/api/v1/add-payrolls`): Hệ thống lưu bản ghi `Payroll` đồng thời tự động cộng dồn số tiền lương thực nhận vào dư nợ lương của nhân viên:

$$\text{Dư nợ lương mới} = \text{Dư nợ lương cũ} + \text{Lương nhận}$$

- Khi **hủy bảng lương** (DELETE `/api/v1/delete-payrolls`): Hệ thống xóa bản ghi `Payroll` đồng thời hoàn trả (trừ đi) dư nợ lương tương ứng:

$$\text{Dư nợ lương mới} = \text{Dư nợ lương cũ} - \text{Lương nhận}$$

3. Nghiệp Vụ Cốt Lõi Tiền Nga (Core Rubber Trading & Inventory)

3.1. Tổng quan về API

Xử lý các phiếu thu mua mủ, quản lý nợ hộ dân, quản lý xuất nhập tồn kho nguyên liệu/thành phẩm và tính toán hao hụt chế biến.

- **Các Endpoint chính:**

- `POST /api/v1/tien-nga/add-daily-purchases` : Ghi nhận phiếu thu mua mủ nước/mủ đông hàng ngày.
- `POST /api/v1/tien-nga/process-debt` : Cấn trừ nợ (thu/chi nợ) cho hộ dân hoặc đối tác.
- `POST /api/v1/tien-nga/process-advance-amount` : Xử lý tạm ứng tiền mặt cho hộ dân (đầu mùa vụ).
- `POST /api/v1/tien-nga/process-deduction-advance-amount` : Khấu trừ tiền tạm ứng vào công nợ mủ.
- `POST /api/v1/tien-nga/process-loss-control` : Tính toán hao hụt mủ chế biến theo lô sản xuất.

3.2. Logic Nghiệp Vụ & Công thức Tính toán

A. Tính Toán Thu Mua Mủ Hàng Ngày (`add-daily-purchases`)

Khi nhận dữ liệu cân mủ của hộ dân, hệ thống tự động thực hiện các phép tính quy đổi:

1. **Khối lượng thực tế (Actual Weight):**

$$\text{Khối lượng thực} = \text{Khối lượng tổng (weight)} - \text{Khối lượng khay/bì (tare_weight)}$$

2. **Khối lượng mủ khô quy đổi (Dry Rubber Weight):**

$$\text{Mủ khô} = \text{Khối lượng thực} \times \frac{\text{Độ mủ (degree)}}{100}$$

3. **Thành tiền cơ bản (Total Amount):**

$$\text{Thành tiền} = \text{Mủ khô} \times \text{Đơn giá mủ (unit_price)}$$

(Nếu là hộ dân được trợ giá `is_subsidized = True`, đơn giá sẽ được cộng thêm giá trị trợ giá `subsidy_price`)

4. **Phân bổ tiền thanh toán và nợ giữ lại:**

- `paid_amount` : Số tiền mặt chi trả ngay.
- `saved_amount` : Số tiền nợ giữ lại để cấn trừ sau.
- **Cập nhật công nợ hộ dân:** Nếu `saved_amount > 0`, hệ thống tự động cộng dồn số tiền này vào công nợ của hộ dân:

$$\text{Công nợ hộ dân mới (total_debt)} = \text{Công nợ cũ} + \text{saved_amount}$$

B. Thuật Toán Cấn Trừ Công Nợ Theo Nguyên Tắc FIFO (`process-debt` với `type_transaction = "chi"`)

Khi trả tiền công nợ cho hộ dân (`amount`), hệ thống tự động trừ dư nợ tổng `customer.total_debt` và thực hiện cấn trừ cho các phiếu thu mua mủ cũ theo thứ tự thời gian tăng dần (First In, First Out):

1. Tìm các phiếu mua mủ (`DailyPurchases`) của hộ dân đó có `saved_amount > 0` .
2. Sắp xếp các phiếu này theo ngày thu mua tăng dần (`day.asc()`).
3. Thực hiện vòng lặp phân bổ số tiền chi trả:
 - Với mỗi phiếu mua mủ i :

$$\text{allocated} = \min(\text{Số tiền còn lại để chi trả}, \text{saved_amount}_i)$$

- Cập nhật số tiền đã trả trên phiếu:

$$\text{paid_amount}_i = \text{paid_amount}_i + \text{allocated}$$

- Giảm số tiền nợ còn lại của phiếu:

$$\text{saved_amount}_i = \text{saved_amount}_i - \text{allocated}$$

- Giảm số tiền quỹ còn lại để phân bổ:

$$\text{Số tiền còn lại} = \text{Số tiền còn lại} - \text{allocated}$$

- Dừng vòng lặp nếu số tiền còn lại bằng 0.

C. Hạn Mức Tạm Ứng Tiền Mặt (`process-advance-amount`)

Để tránh rủi ro nợ xấu, hệ thống áp dụng công thức tính hạn mức tạm ứng tối đa cho hộ dân dựa trên sản lượng giao dịch của mùa vụ trước:

1. **Thời gian mùa vụ trước:**
 - Nếu thời điểm hiện tại từ tháng 5 trở đi: Mùa vụ trước bắt đầu từ **1/5 năm ngoái** đến **ngày cuối cùng của tháng 2 năm nay**.
 - Nếu thời điểm hiện tại trước tháng 5: Mùa vụ trước bắt đầu từ **1/5 của 2 năm trước** đến **ngày cuối cùng của tháng 2 năm ngoái**.
2. **Hạn mức tạm ứng tối đa (Max Cash Advance):**

$$\text{Hạn mức tối đa} = \text{Tổng tiền bán mủ vụ trước} \times \text{Tỷ lệ ứng tối đa (MaxCashAdvance)}$$

(Tỷ lệ `MaxCashAdvance` được định cấu hình trong file `appsettings.json` , thường là 0.5 - tức 50%)

3. **Điều kiện phê duyệt:**
 - Số tiền yêu cầu ứng mới: `cash_advance_requested` .
 - Tổng số tiền tạm ứng sau khi ứng thêm:

$$\text{Tổng ứng mới} = \text{Số tiền đã ứng trước đó} + \text{cash_advance_requested}$$

- **Ràng buộc:** Nếu Tổng ứng mới > Hạn mức tối đa, hệ thống sẽ **từ chối giao dịch** và báo lỗi vượt hạn mức kèm theo báo cáo chi tiết nguyên nhân.

D. Khấu Trừ Tạm Ứng (`process-deduction-advance-amount`)

- Khi tiến hành khấu trừ tạm ứng (`amount`), hệ thống kiểm tra ràng buộc:

Điều kiện: $\text{amount} \leq \text{Cộng nợ hiện tại của khách hàng (total_debt)}$

- Nếu thỏa mãn điều kiện, hệ thống khấu trừ trực tiếp vào số tiền đã tạm ứng:

Tiền tạm ứng mới (`cash_advance`) = Tiền tạm ứng cũ - `amount`

E. Tính Toán Kiểm Soát Hao Hụt Chế Biến (`process-loss-control`)

Dùng để đo lường tỷ lệ hao hụt của mủ từ lúc thu mua (mủ tươi) đến lúc chế biến xong thành phẩm (mủ khô nhập kho):

1. **Khối lượng mủ khô đầu vào (Total Dry Rubber Input):** Tổng mủ khô quy đổi của lô sản xuất từ các phiếu thu mua mủ tươi có cùng `product_code`.
2. **Khối lượng thành phẩm thu hồi đầu ra (Total Import Quantity):** Lấy tổng khối lượng từ bảng giao dịch kho `ProductTransaction` có mã lô `product_code`, ngày giao dịch trùng với ngày hoàn thành dự kiến `estimated_completion`, và loại giao dịch là "Nhập".
3. **Công thức tính tỷ lệ hao hụt (% Loss):**

$$\text{Tỷ lệ hao hụt (\%)} = \text{Round} \left(\frac{\text{Khối lượng mủ khô đầu vào} - \text{Khối lượng thành phẩm đầu ra}}{\text{Khối lượng mủ khô đầu vào}} \times 100, 2 \right)$$

4. Hệ Thống Quản Lý Tín Dụng & Cho Vay (Credit & Interest)

4.1. Tổng quan về API

Quản lý các hợp đồng tín dụng cho vay lấy lãi và theo dõi các kỳ thanh toán lãi của khách hàng.

- **Các Endpoint chính:**
 - `POST /api/v1/credit/add-credits` : Tạo hợp đồng vay tiền mới.
 - `POST /api/v1/credit/add-credit-interests` : Ghi nhận thu lãi định kỳ.
 - `DELETE /api/v1/credit/delete-credit-interests` : Hủy phiếu thu lãi và tính lại nợ lãi.

4.2. Logic Nghiệp Vụ & Ràng buộc

- **Khi thu lãi (`add-credit-interests`):** Khi ghi nhận số tiền trả lãi, hệ thống sẽ thực hiện giảm khoản nợ lãi tương ứng trên

hợp đồng tín dụng đó.

- **Khi hủy giao dịch thu lãi (`delete-credit-interests`)**: Hệ thống xóa bản ghi đóng tiền lãi và **tự động hoàn lại (cộng trả lại)** số nợ lãi chưa thanh toán của hợp đồng để đảm bảo công nợ được tính chính xác như ban đầu.
-

5. Hệ Thống Cho Thuê Bất Động Sản (Real Estate Rental)

5.1. Tổng quan về API

Quản lý danh sách tài sản (nhà, đất, mặt bằng) cho thuê và dòng tiền thu về từ các hợp đồng thuê.

- **Các Endpoint chính:**
 - `POST /api/v1/rental/add-real-estates` : Khai báo tài sản mới.
 - `POST /api/v1/rental/add-rental-payments` : Tạo phiếu thu tiền thuê nhà theo chu kỳ.

5.2. Logic Nghiệp Vụ & Ràng buộc

- **Quản lý trạng thái:** Khi một tài sản được gán vào hợp đồng thuê đang hoạt động, trạng thái của bất động sản đó tự động chuyển từ `"Available"` (Sẵn sàng) sang `"Rented"` (Đang thuê). Khi kết thúc hoặc thanh lý hợp đồng, trạng thái được trả về `"Available"` .
 - **Cảnh báo thanh toán:** Công cụ chạy ngầm `rental_payment_notification_worker` chạy định kỳ hàng ngày sẽ quét các hợp đồng có ngày thanh toán tiếp theo nằm trong phạm vi cảnh báo và gửi thông báo nhắc đóng tiền thuê lên Telegram.
-

6. Phân Hệ Quản Lý Hội/Họ/Biêu/Phường (Rosca System)

6.1. Tổng quan về API

Quản lý các vòng hội (dây hội), danh sách người tham gia (chân hội) và các giao dịch đóng/rút hội định kỳ.

- **Các Endpoint chính:**
 - `POST /api/v1/rosca/add-rosca` : Tạo dây hội (định nghĩa tiền đóng gốc `base_amount`).
 - `POST /api/v1/rosca/add-rosca-contributions` : Ghi nhận giao dịch đóng hội của chân hội (phải gửi số tiền âm).
 - `POST /api/v1/rosca/withdraw-rosca` : Ghi nhận giao dịch rút hội/hốt hội (phải gửi số tiền dương).
 - `DELETE /api/v1/rosca/delete-rosca-contributions` : Xóa giao dịch đóng/rút hội và tự động tính toán lại lịch sử công nợ hội của thành viên.
-

6.2. Logic Nghiệp Vụ & Công thức Tính toán

A. Ràng buộc khi đóng hội hàng kỳ (`add-roscas-contributions`)

1. **Ràng buộc số tiền:** Số tiền đóng hội gửi lên bắt buộc phải là số âm (`amount < 0`).
2. **Ràng buộc đối với Chân hội sống (chưa hết hội - `member.status != "Dead"`):**
 - Người chơi đấu giá kỳ này bằng cách đưa ra mức lãi đề xuất (ví dụ: đấu 500k cho phần hội gốc 5 triệu).
 - Số tiền thực tế người chơi hết được từ chân hội sống khác đóng vào:

$$\text{Số tiền thực đóng} = \text{Tiền hội gốc (base_amount)} - \text{Tiền đấu hội (abs(amount))}$$

- **Kiểm tra biên độ đấu hội:** Số tiền thực đóng này bắt buộc phải nằm trong giới hạn tối thiểu/tối đa cho phép của dây hội:

$$\text{min_bid_amount} \leq \text{Số tiền thực đóng} \leq \text{max_bid_amount}$$

3. **Ràng buộc đối với Chân hội chết (đã hết hội - `member.status == "Dead"`):**
 - Người chơi đã hết hội ở các kỳ trước bắt buộc phải đóng đầy đủ phần hội chết bằng đúng 100% giá trị gốc:

$$\text{Yêu cầu: } \text{abs(amount)} == \text{base_amount}$$

4. **Tránh đóng trùng:** Hệ thống từ chối ghi nhận nếu phát hiện chân hội đó đã có giao dịch đóng tiền trong cùng một ngày.
5. **Cập nhật thông tin Chân hội khi đóng tiền:**
 - Cộng dồn số tiền đã đóng:

$$\text{total_contributed} = \text{total_contributed} + \text{amount} \quad (\text{lưu ý amount là số âm})$$

- Nếu là chân hội chết và hoàn thành đóng kỳ cuối: Tính toán tổng lợi nhuận thu được từ dây hội:

$$\text{total_profit} = \text{total_contributed (số âm)} + \text{total_received (số dương)}$$

B. Ràng buộc khi rút/hết hội (`withdraw-roscas`)

1. **Ràng buộc số tiền:** Số tiền rút hội gửi lên bắt buộc phải lớn hơn 0 (`amount > 0`).
2. **Cập nhật trạng thái Chân hội:**
 - Cộng dồn số tiền đã rút về:

$$\text{total_received} = \text{total_received} + \text{amount}$$

- Chuyển trạng thái thành viên sang Hội chết:

$$\text{member.status} = \text{"Dead"}$$

- Cập nhật lợi nhuận tạm tính tại thời điểm hết hội:

$$\text{total_profit} = \text{total_contributed} + \text{total_received}$$

C. Hoàn trả dữ liệu khi xóa giao dịch hụi (`delete-rosca-contributions`)

Khi xóa một giao dịch đóng/rút hụi, hệ thống tự động tính toán đảo ngược để đưa dữ liệu thành viên về trạng thái đúng:

- Nếu xóa giao dịch đóng hụi (`amount < 0`):

$$\text{total_contributed} = \text{total_contributed} - \text{amount}$$

(Nếu thành viên đang có trạng thái `Dead`, hệ thống tự động cập nhật lại `total\_profit`)

- Nếu xóa giao dịch rút hụi (`amount > 0`):

$$\text{total_received} = \text{total_received} - \text{amount}$$

- Nếu tổng số tiền nhận được quay về ≤ 0 , hệ thống tự động đổi trạng thái thành viên từ `"Dead"` quay về `"Playing"` (Hụi sống).
- Cập nhật lại lợi nhuận của thành viên:

$$\text{total_profit} = \max(\text{total_contributed} + \text{total_received}, 0.0)$$

7. Phân Hệ Quản Lý Phương Tiện & Giấy Tờ (Vehicle & Document Reminders)

7.1. Tổng quan về API

Quản lý đội xe và các tài liệu đi kèm (bảo hiểm xe, giấy đăng kiểm, phù hiệu vận tải).

- Các Endpoint chính:
 - `POST /api/v1/vehicle/add-vehicles` : Khai báo xe mới.
 - `POST /api/v1/vehicle/add-document-reminders` : Thiết lập lịch cảnh báo hết hạn giấy tờ.

7.2. Logic Nghiệp Vụ & Tác vụ tự động

- Cơ chế hoạt động của Scheduler (`document_reminder_worker`):
 - Hàng ngày, tiến trình ngầm tự động quét tất cả các nhắc nhở trong bảng `DocumentReminder` đang ở trạng thái hoạt động.
 - Đối chiếu ngày hết hạn của tài liệu với ngày hiện tại.
 - Nếu số ngày còn lại nằm trong khoảng cảnh báo (thường là 7 ngày hoặc 15 ngày trước khi hết hạn), hệ thống tự động biên soạn tin nhắn Markdown chi tiết (biển số xe, tên giấy tờ, ngày hết hạn thực tế) và gửi trực tiếp tới nhóm Telegram được chỉ định của tài xế và quản lý đội xe.
-

8. Tích Hợp Telegram (Telegram Bot Integration)

8.1. Tổng quan về API

Tích hợp trực tiếp bot Telegram với các sự kiện thay đổi dữ liệu trong hệ thống.

- **Các Endpoint chính:**

- `POST /api/v1/telegram/notify` : API gửi thông tin sự kiện và kích hoạt gửi tin Telegram.
- `POST /api/v1/telegram-group-mappings/` : Cấu hình phân tuyến nhóm nhận tin nhắn.

8.2. Logic Phân Tuyến Tin Nhắn (Notification Routing)

1. Khi có bất kỳ hành động Thay đổi dữ liệu nào (`CREATE` , `UPDATE` , `DELETE`) từ các phân hệ chính (như chấm công, tiền lương, hóa đơn, tồn kho), hệ thống sẽ gọi đến hàm:
`await notify_telegram_group(db, action, module_key, details, performer)`
 2. Hệ thống tìm kiếm trong bảng cấu hình `TelegramGroupMapping` theo khóa `mapping_type` tương ứng với `module_key` (ví dụ: `salaries` , `customers` , `loss_controls` , `inventories`).
 3. Lấy ra **Chat ID** tương ứng được cấu hình.
 4. Bot Telegram sử dụng kết nối Pyrogram đang chạy để gửi tin nhắn Markdown có cấu trúc đẹp mắt tới nhóm làm việc thích hợp.
-

9. Phân Hệ Quản Lý Dự Án (Business / Projects)

9.1. Tổng quan về API

Khai báo và quản lý tiến độ, nguồn vốn và nhân sự của các dự án công trình xây dựng hoặc đầu tư lớn.

- **Các Endpoint chính:**

- `POST /api/v1/business/create_project` : Khởi tạo thông tin dự án.
- `POST /api/v1/telegram/create_project_member` : Liên kết nhân viên chịu trách nhiệm dự án với tài khoản Telegram định danh.

9.2. Logic Vận Hành

- **Định danh telegram:** Khi liên kết thành viên dự án, hệ thống lưu trữ UUID của dự án cùng thông tin Telegram Chat ID của người dùng. Điều này hỗ trợ cho việc phân quyền phê duyệt các đề xuất chi ngân sách dự án thông qua tương tác trực tiếp bằng nút bấm trên Chatbot Telegram sau này.